

- expire after an inordinately long time) will also help mitigate the threat.
5. If possible, don't store user-names and passwords in the browser's password manager (for example, see <http://lwn.net/Articles/211875/>).

Click-jacking

Click-jacking is an attack involving embedded objects on a maliciously crafted Web page. Using framed content, or that from Flash, Silverlight, or Java, the attacker places a transparent or invisible click button beneath the mouse, so that whenever the user clicks on something they see on the page, the user is also clicking to an unseen website that may contain malicious code. The attack can also take advantage of dynamic HTML and CSS (Cascading Style Sheets) code for further disguise. The difference between CSRF and click-jacking is that in CSRF, the victim's browser performs the attack (loading the state-changing URL directly) without the victim clicking to launch it, while in click-jacking, the user actually interacts with something, but the action is 'hijacked' by placing a layer between the user and the page element that launches a legitimate action.

For developers

1. The best defence against CSRF attacks is unpredictable tokens, a piece of data that the server can use to validate the request, and which an attacker can't guess. For example, an important request could contain a digest of the user's session credential, which is different for every user. And, for a little extra security, add a timestamp to the token, to limit the window of opportunity, as shown in the POST body below:

```
POST http://fictitiousbank/transfer.cgi HTTP/1.1
Host: fictitiousbank
User-Agent: Mozilla/5.0 (Windows; U; Windows NT 5.1; de; rv:1.9)
Gecko/2008052906
Firefox/3.6.2
Cookie: PHPSESSID=7757A0D8766d45NFJ323875JBKBF from=3536702
1&to=48412334&amount=-05672010&token=48E03EF45T443W2K4IC567HY433
40D446&timestamp=1184001456
```

The tokens used should also be cryptographically very strong.

2. Limit the time for which the user's credentials are valid. By enforcing inactivity time-outs, you reduce chances of CSRF attacks.
3. Password re-verification should be given priority over single-sign on. In this method, the users must type in their passwords again when accessing particularly critical functions.
4. Switching over to URL rewriting is not recommended since, when URLs contain session tokens, the tokens could be leaked via proxy-logs/referrers. Moreover, they are also not very helpful against local/stored attacks, since all URLs produced by the application contain the token.
5. Do not rely (solely) on referrer checking, as techniques exist to selectively create HTTP requests without referrers.

6. For protection against local/stored attacks, you should mirror all foreign content, and don't allow arbitrary URLs in your Web application. Moreover, local attacks can be mitigated if you only serve images from your own servers (like for social sites and forums) and also don't allow users to store arbitrary data on your servers.
7. When building defences against CSRF, you must eliminate XSS vulnerabilities.
8. Throw as many roadblocks at the attacker as possible, including customised error messages, checks on HTTP referrer headers and Web application firewalls.
9. Use CAPTCHAs, especially for important transactions, to check whether there's a human being at the other end, and not an automated attack.
10. Harden the intranet websites, apply security patches and updates, and change default passwords.

Tools of the security trade

There are Web scanners available, both commercial and FOSS, to test websites for CSRF, but it's very rare that they find CSRF code, given the complexity of CSRF attacks. However, there is some FOSS available to make security work easy:

1. OWASP CSRF tester: This is basically a JavaEE filter that implements the synchroniser token pattern to mitigate the risk of CSRF attacks. Its documentation can be found at http://www.owasp.org/index.php/Category:OWASP_CSRFTester_Project. A PHP implementation of this CSRF guard is available at www.owasp.org/index.php/PHP_CSRF_Guard and a .NET implementation at www.owasp.org/index.php/.Net_CSRF_Guard.
2. RequestRodeo is an HTTP proxy written in Python, using the Twisted framework, OpenSSL and SQLite. It protects its user against CSRF. It can be found at <http://savannah.nongnu.org/projects/requestrodeo/>.

For more detailed information on CSRF, and more defences against it, don't forget to visit the resources below. We will deal with other dangerous attacks on Web applications and Apache in the next article. Always remember: know hacking, but no hacking. 

Further reading

- http://packetstormsecurity.org/papers/attack/Using_XSS_to_bypass_CSRF_protection.pdf
- Visit <http://www.seccologic.org/>
- http://www.owasp.org/index.php/Cross-Site_Request_Forgery_%28CSRF%29_Prevention_Cheat_Sheet
- <http://www.whitehatsec.com/> contains good information both on XSS and CSRF and other attacks.

By: Arpit Bajpai

The author is a FOSS enthusiast, and loves to troubleshoot in the information security domain, while exploring new exploits and system vulnerabilities. You can send your queries and constructive feedback on this article to abajpai75 AT yahoo DOT com.